

Universidad de Las Palmas de Gran Canaria

Grado en Ingeniería Informática

MazeBall: Juego de laberintos con control por gestos de mano

Memoria del proyecto de Visión por Computador

Autores: Juan Boissier García / Carlos Ruano Ramos
Fecha: 12/01/2026

Índice general

Resumen	II
1. Introducción	1
1.1. Motivación y argumentación del trabajo	1
1.2. Objetivo de la propuesta	1
2. Descripción técnica del trabajo	2
2.1. Visión por computador y seguimiento de manos	2
2.2. Detección de gestos y formas de mano	2
2.3. Motores de videojuegos y Godot	2
2.4. Arquitectura general del sistema	2
2.5. Módulo de visión y gestos	2
2.6. Integración con Godot y control del juego	3
2.7. Procesamiento de vídeo y detección de mano	3
2.8. Cálculo de gestos y orientación	3
2.9. Comunicación UDP con Godot	3
2.10. Lógica de juego en Godot	3
3. Fuentes y tecnologías utilizadas	4
3.1. Lenguajes y librerías	4
3.2. Herramientas de desarrollo	4
3.3. Fuentes de documentación y aprendizaje	4
4. Conclusiones y propuestas de ampliación	5
4.1. Conclusiones	5
4.2. Propuestas de ampliación	5
4.3. Herramientas deseadas	5
5. Diario de reuniones del grupo	6
5.1. Resumen cronológico	6
6. Créditos de materiales no originales	7
6.1. Recursos gráficos y de audio	7
6.2. Código y ejemplos externos	7

Resumen

MazeBall es un juego de puzles y plataformas desarrollado en el motor **Godot** que explora formas alternativas de interacción hombre-máquina mediante visión por computador. La propuesta combina mecánicas clásicas de laberintos con un sistema de control híbrido que integra entrada tradicional y reconocimiento gestual en tiempo real.

El núcleo jugable consiste en guiar una bola a través de circuitos con obstáculos, interruptores y plataformas móviles, resolviendo retos de precisión y coordinación espacial. Para ello, el jugador puede emplear tanto teclado y ratón como gestos de la mano capturados desde la webcam, lo que permite adaptar el control a distintos contextos y preferencias.

El sistema de control por gestos se basa en MediaPipe y OpenCV para detectar la posición y la pose de la mano, mapeando estos datos a acciones de juego como inclinar el escenario, activar mecanismos o interactuar con elementos del entorno. Esta integración convierte a MazeBall en un entorno experimental donde se evalúa la viabilidad del reconocimiento de manos como interfaz de entrada para videojuegos de puzles y plataformas.

Capítulo 1

Introducción

1.1. Motivación y argumentación del trabajo

Los videojuegos han incorporado progresivamente interfaces naturales de usuario, como el control por movimiento, el reconocimiento de voz o el seguimiento de manos, con el objetivo de lograr una interacción más intuitiva y accesible. El uso de técnicas de visión por computador y bibliotecas como MediaPipe permite detectar la posición y la pose de la mano en tiempo real, facilitando nuevos esquemas de control en juegos 2D y 3D. Estas interfaces gestuales tienen potencial para aplicarse en entornos educativos y en ejercicios de control psicomotor, aprovechando el componente lúdico del videojuego para reforzar el aprendizaje y la rehabilitación.

1.2. Objetivo de la propuesta

Los objetivos principales del proyecto son:

- Diseñar e implementar un juego tipo laberinto y plataformas en Godot con física de bola.
- Integrar un sistema de reconocimiento de mano basado en MediaPipe y OpenCV que detecte poses y gestos específicos.
- Transmitir en tiempo real la información de la mano desde Python al juego mediante sockets UDP para controlar mecánicas de juego concretas.
- Evaluar la jugabilidad y robustez del sistema de control por gestos en distintas condiciones de uso.

Capítulo 2

Descripción técnica del trabajo

2.1. Visión por computador y seguimiento de manos

En este proyecto se emplea un flujo de visión por computador que parte de la captura de vídeo en tiempo real mediante una webcam, procesando cada fotograma con OpenCV y MediaPipe para extraer puntos clave (*landmarks*) de la mano. MediaPipe proporciona un modelo de *Hand Landmarker* que devuelve 21 puntos tridimensionales por mano, permitiendo analizar la postura de los dedos y la orientación de la palma.

2.2. Detección de gestos y formas de mano

A partir de los *landmarks* es posible derivar características geométricas como vectores dedo-palma, proyección sobre la normal de la palma y ángulos entre articulaciones, que se usan para clasificar gestos. En el proyecto se distinguen gestos como *index*, *rock*, *C* y *peace*, en función del estado extendido o flexionado de cada dedo y de los ángulos entre segmentos.

2.3. Motores de videojuegos y Godot

Godot es un motor de videojuegos libre que soporta escenas jerárquicas, scripting en GDScript y motores de física 2D y 3D, lo que facilita la implementación de un juego de laberintos con una bola controlada por fuerzas y colisiones. El motor permite integrar entradas personalizadas que, en este caso, se enriquecen con información proveniente del sistema de reconocimiento de mano en Python.

2.4. Arquitectura general del sistema

La arquitectura se compone de dos procesos principales: un módulo de visión por computador en Python que captura y analiza la mano, y el juego *MazeBall* implementado en Godot que recibe y utiliza estos datos. La comunicación entre ambos componentes se realiza mediante sockets UDP, enviando mensajes en formato JSON con información de posición, tamaño, gesto y orientación de la mano.

2.5. Módulo de visión y gestos

El módulo de visión abre la cámara con OpenCV, procesa los fotogramas mediante MediaPipe Tasks y mantiene un estado suavizado por cada mano detectada. Para cada mano se calcula un *bounding box* rotado a partir de puntos de los dedos relevantes y se obtienen métricas como el centro, las longitudes del eje principal y el ángulo de orientación.

2.6. Integración con Godot y control del juego

En el lado de Godot, un script recibe los paquetes UDP, decodifica el JSON y mapea las variables recibidas (posición de la mano, gesto y ángulo) a acciones de juego como empujar la bola o activar interruptores. El diseño del juego combina entradas tradicionales de teclado/ratón con eventos derivados de gestos de mano para ofrecer un esquema de control híbrido.

2.7. Procesamiento de vídeo y detección de mano

El fichero `opencv2.py` implementa la inicialización de la cámara, la creación del *HandLandmarker* en modo *LIVE_STREAM* y un bucle que captura, voltea y convierte los fotogramas a formato compatible con MediaPipe. El resultado de la detección se gestiona mediante un *callback* que actualiza una estructura global con los últimos *landmarks* de cada mano detectada.

2.8. Cálculo de gestos y orientación

Sobre los *landmarks* se construyen vectores palma-dedo y se calcula la normal de la palma, determinando si cada dedo está extendido o flexionado según su proyección y ángulos articulares. La clasificación de la forma de la mano deriva etiquetas como *index*, *rock*, *C* o *peace*, que posteriormente se usan en la lógica del juego.

2.9. Comunicación UDP con Godot

Periódicamente, el módulo Python empaqueta en un diccionario información de cada mano, incluyendo coordenadas del centro, tamaño del *bounding box*, etiqueta de gesto, ángulo y banderas adicionales como la inversión. Este diccionario se serializa en JSON y se envía por UDP a la dirección y puerto configurados de Godot, permitiendo que el motor utilice estos datos casi en tiempo real.

2.10. Lógica de juego en Godot

En Godot, la bola protagonista se controla mediante fuerzas o cambios de velocidad que se combinan con la entrada estándar de teclado y los gestos de mano recibidos. Determinados gestos pueden mapearse a acciones como atraer o repeler la bola, accionar plataformas móviles o activar interruptores en los niveles de *MazeBall*.

Capítulo 3

Fuentes y tecnologías utilizadas

3.1. Lenguajes y librerías

El proyecto utiliza Python 3.8+ junto con las bibliotecas `mediapipe`, `opencv-python` y `numpy` para el procesamiento de vídeo y geometría de la mano. El juego *MazeBall* está desarrollado en Godot, con scripts en GDScript para la lógica del juego y escenas que definen los niveles de laberinto y plataformas.

3.2. Herramientas de desarrollo

Como entorno de desarrollo se emplean un editor de código compatible con Python y Godot, el propio editor del motor Godot y herramientas de control de versiones como Git para gestionar la evolución del proyecto. Además, se han utilizado entornos virtuales de Python para aislar dependencias y facilitar la instalación en distintas máquinas.

3.3. Fuentes de documentación y aprendizaje

Las principales fuentes de documentación han sido la referencia oficial de MediaPipe para el seguimiento de manos, la documentación de OpenCV y los manuales y tutoriales del motor Godot. También se han consultado recursos adicionales sobre control gestual y diseño de interfaces naturales aplicadas a videojuegos.

Capítulo 4

Conclusiones y propuestas de ampliación

4.1. Conclusiones

El proyecto demuestra que es viable integrar un sistema de reconocimiento de manos en tiempo real con un videojuego desarrollado en Godot, utilizando MediaPipe y OpenCV como base del módulo de visión. La jugabilidad de *MazeBall* se ve enriquecida por la posibilidad de controlar la bola y los elementos del escenario mediante gestos, aunque el rendimiento y la robustez dependen de factores ambientales y de la precisión de la detección.

4.2. Propuestas de ampliación

Entre las posibles extensiones del trabajo se encuentran:

- Incorporar modelos de seguimiento 3D más avanzados para estimar profundidad y distancia de la mano a la cámara.
- Diseñar un sistema de personalización de gestos para que el usuario pueda entrenar y asignar sus propias poses a acciones del juego.
- Ampliar el juego añadiendo más niveles y posibilidades de mezclar gestos.

4.3. Herramientas deseadas

Como reflexión final, habría sido muy útil que Godot ofreciera soporte nativo para acceder a la cámara y procesar directamente el flujo de vídeo con MediaPipe, evitando así la necesidad de emplear procesos externos y comunicación mediante sockets UDP para el intercambio de datos.

Capítulo 5

Diario de reuniones del grupo

En esta sección se recoge un resumen de las reuniones mantenidas por el grupo, con la fecha, los asistentes y los acuerdos principales.

5.1. Resumen cronológico

- **10/12/2025:** Definición de la idea del proyecto, reparto inicial de tareas (módulo de visión, integración con Godot, diseño de niveles).
- **15/12/2025:** Primera integración básica de detección de mano con el juego, revisión de problemas de latencia y estabilidad.
- **28/12/2025:** Afinado de la clasificación de gestos y pruebas de jugabilidad con usuarios internos.
- **05/01/2026:** Preparación de la memoria, grabación de material de demostración y cierre de tareas pendientes.

Capítulo 6

Créditos de materiales no originales

En este capítulo se listan los recursos externos utilizados en el proyecto, indicando su autoría y licencia.

6.1. Recursos gráficos y de audio

- Sprites de la bola y elementos del escenario procedentes de Omaira Boissier García, que diseñó todos los sprites del juego.
- Efectos de sonido y música de fondo procedentes de FreeSound, con atribución según los términos de la licencia.

6.2. Código y ejemplos externos

Además de las propias implementaciones, se han consultado ejemplos y fragmentos de código de la documentación oficial de MediaPipe y OpenCV, así como tutoriales orientados a la detección de manos y al control de aplicaciones interactivas mediante gestos.

Bibliografía

- [1] Asistente de ia para apoyo en el desarrollo. Herramienta de inteligencia artificial basada en lenguaje. Utilizada durante el desarrollo del proyecto.
- [2] Documentación oficial de godot engine. <https://docs.godotengine.org/>. Consultado el 9 de enero de 2026.
- [3] Mediapipe documentation. <https://developers.google.com/mediapipe>. Consultado el 9 de enero de 2026.
- [4] Tutorial de youtube sobre integración de cámara en godot. <https://www.youtube.com/watch?v=2TZeGscbjeE&t=491s>. Consultado el 2 de enero de 2026.

Como referencia se ha utilizado la documentación oficial de Godot [2] y MediaPipe [3], así como un asistente de IA [1] y tutoriales de YouTube [4].